

full-project-python

December 1, 2025

1 Egypt Real Estate Listings

1.1 Abstract

This project focuses on analyzing and predicting real estate prices in Egypt using a dataset containing thousands of property listings. The dataset includes features such as location, area, number of rooms, bathrooms, and property type, along with price information. The goal of this project is to build a complete data analysis pipeline that automates data cleaning, preprocessing, visualization, and modeling using Python libraries like NumPy, Pandas, Matplotlib, and Seaborn. Insights from this dataset can help identify market trends, understand factors affecting housing prices, and support better real estate investment decisions.

1.2 Dataset Summary

The dataset contains 19924 rows and 11 features & multiple property listings across different Egyptian cities and regions, with details about each property's characteristics and pricing. It includes numerical features such as area and price, as well as categorical attributes like city, property type, and furnishing status. The dataset contains around several thousand rows and multiple columns describing each listing. Key problems detected include missing values, non-numeric symbols within numeric fields (e.g., "1,200 EGP", "250 sqm"), inconsistent text formats, duplicated rows, and outliers in property prices and sizes. These issues will be resolved through preprocessing before performing exploratory analysis and machine learning modeling to predict property prices.

2 Data Understanding & Profiling

2.1 Step 1: import packages to use

```
[1]: # import pandas as pd
      # import numpy as np
      # import matplotlib.pyplot as plt
      # import seaborn as sns
      # from pandas_profiling import ProfileReport
      # %matplotlib inline
```

```
[2]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import joblib
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pandas_profiling import ProfileReport
%matplotlib inline
```

<IPython.core.display.HTML object>

C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\4205163976.py:9:
DeprecationWarning: `import pandas\_profiling` is going to be deprecated by April 1st. Please use `import ydata\_profiling` instead.
from pandas_profiling import ProfileReport

```
[3]: pd.set_option('display.max_columns', None)
pd.set_option('display.width', 120)
```

2.2 Step 2: Load the dataset

```
[4]: # Load dataset
df = pd.read_csv("egypt_real_estate_listings.csv", encoding='utf-8')

# Display basic info about the dataset
print(" Dataset loaded successfully!")
print("Shape (rows, columns):", df.shape)
df.head()
```

Dataset loaded successfully!
Shape (rows, columns): (19924, 11)

```
[4]:
```

	description \	url	price
0	https://www.propertyfinder.eg/en/plp/buy/chale... IN EL GOUNA WITH A PRIME LOCATION...	8,000,000	OWN A CHALET
1	https://www.propertyfinder.eg/en/plp/buy/villa... villa with immediate delivery in C...	25,000,000	For sale, a
2	https://www.propertyfinder.eg/en/plp/buy/chale... payment of EGP 1,513,000, a fully ...	15,135,000	With a down
3	https://www.propertyfinder.eg/en/plp/buy/apart... apartment in New Cairo with a minimal d...	12,652,000	Own an
4	https://www.propertyfinder.eg/en/plp/buy/villa... Granville\nLocation: Fifth Settlement...	45,250,000	Project:

	size	bedrooms	bathrooms	\	location	type	
0	68 sqm	1+	Maid	1	Swan Lake Gouna, Al Gouna, Hurghada, Red Sea	Chalet	732 sqft /
1	220 sqm	4		4	Karmell, New Zayed City, Sheikh Zayed City, Giza	Villa	2,368 sqft /
2	118 sqm	2		2	Azha North, Ras Al Hekma, North Coast	Chalet	1,270 sqft /
3	166 sqm	3		2	Taj City, 5th Settlement Compounds, The 5th Se...	Apartment	1,787 sqft /
4	400 sqm	7		7	Granville, New Capital City, Cairo	Villa	4,306 sqft /

	available_from	payment_method	down_payment
0	31 Aug 2025	Cash	1,200,000 EGP
1	2 Sep 2025	Cash	2,100,000 EGP
2	19 Aug 2025	Cash	1,513,000 EGP
3	26 Aug 2025	Installments	1,260,000 EGP
4	2 Sep 2025	Cash	2,262,500 EGP

2.3 Step 3: General Dataset Overview

```
[5]: # Display general information about the dataset (columns, data types, non-null
      ↪ counts)
df.info()

# Show statistical summary for both numeric and categorical columns
print("\nStatistical Summary (Numeric & Categorical):\n")
display(df.describe(include='all').T)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19924 entries, 0 to 19923
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   url                    19924 non-null  object
1   price                  19385 non-null  object
2   description             19846 non-null  object
3   location                19833 non-null  object
4   type                   19847 non-null  object
5   size                   19847 non-null  object
6   bedrooms               19780 non-null  object
7   bathrooms              19784 non-null  object
8   available_from         19261 non-null  object
9   payment_method         19383 non-null  object
10  down_payment           5445 non-null   object
dtypes: object(11)
```

memory usage: 1.7+ MB

Statistical Summary (Numeric & Categorical):

	count	unique	top
freq			
url	19924	19924	https://www.propertyfinder.eg/en/plp/buy/chale...
price	19385	4286	10,000,000
description	19846	18130	Please Note Before Reading:\nThis is one of ma...
location	19833	1535	Marassi, Sidi Abdel Rahman, North Coast
type	19847	17	Apartment
size	19847	683	1,507 sqft / 140 sqm
bedrooms	19780	18	3
bathrooms	19784	16	3
available_from	19261	353	1 Sep 2025
payment_method	19383	2	Cash
down_payment	5445	2030	100,000 EGP

2.4 Step 4: Check for Duplicates, Empty Columns, and Missing Values Summary

```
[6]: # Check for duplicate rows
duplicates = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicates}")

# Check for completely empty columns (all values are NaN)
empty_columns = [col for col in df.columns if df[col].isna().all()]
print(f"Completely empty columns: {empty_columns}")

# Count missing values per column
missing_values = df.isna().sum().sort_values(ascending=False)
print("\nMissing Values per Column:\n", missing_values)

# Calculate percentage of missing values per column
```

```
missing_percentage = (df.isna().mean() * 100).round(2).
↳sort_values(ascending=False)
print("\nPercentage of Missing Values:\n", missing_percentage)
```

Number of duplicate rows: 0

Completely empty columns: []

Missing Values per Column:

down_payment	14479
available_from	663
payment_method	541
price	539
bedrooms	144
bathrooms	140
location	91
description	78
type	77
size	77
url	0

dtype: int64

Percentage of Missing Values:

down_payment	72.67
available_from	3.33
payment_method	2.72
price	2.71
bedrooms	0.72
bathrooms	0.70
location	0.46
description	0.39
type	0.39
size	0.39
url	0.00

dtype: float64

2.5 Step 5: Check Unique Values and Cardinality of Each Column

```
[7]: # Count the number of unique values in each column
unique_counts = df.nunique().sort_values(ascending=False)
print("Unique values per column:\n", unique_counts)

# Display columns with very low or very high uniqueness
high_cardinality = unique_counts[unique_counts > (0.9 * len(df))]
low_cardinality = unique_counts[unique_counts <= 5]

print("\nColumns with very high uniqueness (possible IDs or URLs):\n",
↳high_cardinality)
```

```
print("\nColumns with very low uniqueness (might be categorical with few
↪values):\n", low_cardinality)
```

Unique values per column:

```
url          19924
description  18130
price        4286
down_payment 2030
location     1535
size         683
available_from 353
bedrooms     18
type         17
bathrooms    16
payment_method 2
dtype: int64
```

Columns with very high uniqueness (possible IDs or URLs):

```
url          19924
description  18130
dtype: int64
```

Columns with very low uniqueness (might be categorical with few values):

```
payment_method 2
dtype: int64
```

2.6 Step 6: Generate a full automated data profiling report for deeper exploration of the dataset.

```
[8]: from ydata_profiling import ProfileReport

profile = ProfileReport(df, title="data Report", explorative=True)

profile.to_notebook_iframe()
```

```
Summarize dataset:  0%|          | 0/5 [00:00<?, ?it/s]
100%|          | 11/11 [00:05<00:00, 1.93it/s]
Generate report structure:  0%|          | 0/1 [00:00<?, ?it/s]
Render HTML:  0%|          | 0/1 [00:00<?, ?it/s]
<IPython.core.display.HTML object>
```

3 Data Cleaning Process

3.1 This section focuses on cleaning, fixing, and preparing the dataset for analysis and modeling.

3.2 Step 1: Standardize Column Names

```
[9]: # Convert all column names to lowercase and replace spaces or special characters
df.columns = (
    df.columns
    .str.strip()          # Remove leading/trailing spaces
    .str.lower()          # Convert to lowercase
    .str.replace(' ', '_') # Replace spaces with underscores
    .str.replace('-', '_') # Replace hyphens with underscores
)
```

```
[10]: # Display the cleaned column names
df.columns.tolist()
```

```
[10]: ['url',
      'price',
      'description',
      'location',
      'type',
      'size',
      'bedrooms',
      'bathrooms',
      'available_from',
      'payment_method',
      'down_payment']
```

3.3 Step 2: Clean numeric columns before filling missing values

```
[11]: # Step: Extract and clean property size properly

# Ensure column is string
df['size'] = df['size'].astype(str)

# Extract sqm if available
df['size_sqm'] = df['size'].str.extract(r'/(s*(\d+\.?\d*)\s*sqm)')[0]

# Extract sqft if sqm is missing
mask_missing = df['size_sqm'].isna()
df.loc[mask_missing, 'size_sqm'] = (
    df.loc[mask_missing, 'size']
    .str.extract(r'(\d+\.?\d*)\s*sqft')[0]
    .astype(float) * 0.093 # Convert sqft → sqm
)
```

```

# Convert to float
df['size_sqm'] = df['size_sqm'].astype(float)

# Drop the old 'size' column
df.drop(columns=['size'], inplace=True)

# Verify result
df[['price', 'down_payment', 'size_sqm']].head()

```

```

[11]:
      price  down_payment  size_sqm
0   8,000,000  1,200,000 EGP      68.0
1  25,000,000  2,100,000 EGP     220.0
2  15,135,000  1,513,000 EGP     118.0
3  12,652,000  1,260,000 EGP     166.0
4  45,250,000  2,262,500 EGP     400.0

```

3.4 Step 3: Add derived columns

```

[12]: #
df['price'] = (
    df['price']
    .astype(str)
    .str.replace(',', '', regex=False)      #
    .str.replace('EGP', '', regex=False)    #      EGP
    .str.replace(r'[^d.]', '', regex=True)  #
)

df['down_payment'] = (
    df['down_payment']
    .astype(str)
    .str.replace(',', '', regex=False)
    .str.replace('EGP', '', regex=False)
    .str.replace(r'[^d.]', '', regex=True)
)

```

```

[13]: #      price  down_payment
df['price'] = pd.to_numeric(df['price'], errors='coerce')
df['down_payment'] = pd.to_numeric(df['down_payment'], errors='coerce')
#
df["down_payment_ratio"] = (df["down_payment"] / df["price"]).round(2)
#              )      1      0.01)
invalid_ratio = (df['down_payment_ratio'] < 0.01) | (df['down_payment_ratio'] > 0.8)

```

```

[14]: df.head()

```



```
[14]:
```

	url	price	
description \			
0	https://www.propertyfinder.eg/en/plp/buy/chale...	8000000.0	OWN A CHALET IN EL GOUNA WITH A PRIME LOCATION...
1	https://www.propertyfinder.eg/en/plp/buy/villa...	25000000.0	For sale, a villa with immediate delivery in C...
2	https://www.propertyfinder.eg/en/plp/buy/chale...	15135000.0	With a down payment of EGP 1,513,000, a fully ...
3	https://www.propertyfinder.eg/en/plp/buy/apart...	12652000.0	Own an apartment in New Cairo with a minimal d...
4	https://www.propertyfinder.eg/en/plp/buy/villa...	45250000.0	Project: Granville\nLocation: Fifth Settlement...

	location	type	bedrooms
bathrooms available_from payment_method \			
0	Swan Lake Gouna, Al Gouna, Hurghada, Red Sea	Chalet	1+ Maid
1	31 Aug 2025 Cash		
1	Karmell, New Zayed City, Sheikh Zayed City, Giza	Villa	4
4	2 Sep 2025 Cash		
2	Azha North, Ras Al Hekma, North Coast	Chalet	2
2	19 Aug 2025 Cash		
3	Taj City, 5th Settlement Compounds, The 5th Se...	Apartment	3
2	26 Aug 2025 Installments		
4	Granville, New Capital City, Cairo	Villa	7
7	2 Sep 2025 Cash		

	down_payment	size_sqm	down_payment_ratio
0	1200000.0	68.0	0.15
1	2100000.0	220.0	0.08
2	1513000.0	118.0	0.10
3	1260000.0	166.0	0.10
4	2262500.0	400.0	0.05

```
[15]: #
df['available_from'] = pd.to_datetime(df['available_from'], errors='coerce')

#
df['year'] = df['available_from'].dt.year

#
# (January, February, ...)
df['month'] = df['available_from'].dt.strftime('%B')

df['year'] = df['year'].astype('Int64')
df['month'] = df['month'].astype(str)
```

4 step 4: Fix unrealistic values

4.1 Handle unrealistic prices

```
[16]: df.loc[(df['price'] < 500_000) | (df['price'] > 100_000_000), 'price'] = np.nan
df['price'] = df['price'].fillna(df['price'].median())
```

4.2 Validate down_payment relative to price

```
[17]: df.loc[
    (df['down_payment'] <= 0) |
    (df['down_payment'] > df['price']) |
    (df['down_payment'] < 0.02 * df['price']) |
    (df['down_payment'] > 0.7 * df['price']),
    'down_payment'
] = np.nan
```

4.3 Fix unrealistic down_payment_ratio

```
[18]: #           price  down_payment  NaN
df.loc[df['down_payment_ratio'].isna(), 'down_payment_ratio'] = (
    df['down_payment'] / df['price']
)
median_ratio = df['down_payment_ratio'].median()
df['down_payment_ratio'] = df['down_payment_ratio'].fillna(median_ratio)

df.loc[
    (df['down_payment'].isna()) | (df['down_payment'] <= 10000),
    'down_payment'
] = df['price'] * df['down_payment_ratio']

print(df['down_payment_ratio'].isna().sum())
print(df['down_payment'].isna().sum())
```

0
0

```
[19]: print(df['down_payment_ratio'].describe())
print(df['size_sqm'].describe())
```

count	19924.000000
mean	0.111483
std	0.074290
min	0.000000
25%	0.100000

```

50%          0.100000
75%          0.100000
max          0.500000
Name: down_payment_ratio, dtype: float64
count      19847.000000
mean       202.597664
std        126.661137
min         0.186000
25%        125.000000
50%        170.000000
75%        239.000000
max         985.000000
Name: size_sqm, dtype: float64

```

4.4 Validate bedroom & bathroom counts

```

[20]: # Ensure numeric columns are actually numeric
df['bedrooms'] = pd.to_numeric(df['bedrooms'], errors='coerce')
df['bathrooms'] = pd.to_numeric(df['bathrooms'], errors='coerce')

# Now safe to validate
df.loc[(df['bedrooms'] < 1) | (df['bedrooms'] > 7), 'bedrooms'] = np.nan
df.loc[(df['bathrooms'] < 1) | (df['bathrooms'] > 7), 'bathrooms'] = np.nan

```

4.5 Step 5: Handle Missing Values

```

[21]: # Check percentage of missing values
missing_percent = df.isnull().mean() * 100
print("Percentage of Missing Values:\n", missing_percent)

# Fill missing numeric values
df['price'] = df['price'].fillna(df['price'].median())
df['size_sqm'] = df['size_sqm'].fillna(df['size_sqm'].median())
df['down_payment'] = df['down_payment'].fillna(df['down_payment'].median())

# Fill missing categorical values
df['bedrooms'] = df['bedrooms'].fillna(df['bedrooms'].mode()[0])
df['bathrooms'] = df['bathrooms'].fillna(df['bathrooms'].mode()[0])
df['available_from'] = df['available_from'].fillna('Unknown')
df['payment_method'] = df['payment_method'].fillna(df['payment_method'].
    ↪mode()[0])
df['location'] = df['location'].fillna('Unknown')
df['type'] = df['type'].fillna('Unknown')
df['description'] = df['description'].fillna('No description')

# Fill missing in new time columns (year and month_name)
df['year'] = df['year'].fillna(df['year'].mode()[0])

```

```
df['month'] = df['month'].fillna('Unknown')

# Ensure year is integer (no .0 in Excel)
df['year'] = df['year'].astype('Int64')
df['month'] = df['month'].astype(str)
```

Percentage of Missing Values:

url	0.000000
price	0.000000
description	0.391488
location	0.456736
type	0.386469
bedrooms	49.126681
bathrooms	1.525798
available_from	3.327645
payment_method	2.715318
down_payment	0.000000
size_sqm	0.386469
down_payment_ratio	0.000000
year	3.327645
month	0.000000

dtype: float64

[22]: *# Step X: Clean and fill missing values in 'month' column*

```
# : NaN
df['month'] = df['month'].replace(['nan', 'NaN', 'None', 'Unknown', '', ' '],
    ↪ np.nan)

# ) (
month_mode = df['month'].mode()[0]

#
df['month'] = df['month'].fillna(month_mode)

print("Missing values in 'month' after filling:", df['month'].isnull().sum())
print("Most frequent month (used to fill NaNs):", month_mode)
```

Missing values in 'month' after filling: 0

Most frequent month (used to fill NaNs): August

[23]: *# Verify changes*
df.isnull().sum()

```
[23]: url          0
      price        0
      description  0
```

```

location          0
type              0
bedrooms         0
bathrooms        0
available_from    0
payment_method    0
down_payment      0
size_sqm         0
down_payment_ratio 0
year             0
month            0
dtype: int64

```

4.6 Remove unnecessary columns

```

[24]: df = df.drop(columns=['url'], errors='ignore')
      df = df.drop(columns=['description'], errors='ignore')
      df = df.drop(columns=['available_from'], errors='ignore')

```

4.7 Step 6: Clean and normalize text columns

```

[25]: text_columns = ['location', 'type', 'payment_method']

for col in text_columns:
    df[col] = (
        df[col]
        .astype(str)                                # Ensure the column is of string
        ↪type
        .str.strip()                                # Remove leading/trailing spaces
        .str.replace(r'\n', ' ', regex=True)         # Replace line breaks with
        ↪spaces
        .str.replace(r'\r', '', regex=True)           # Remove carriage returns
        .str.replace(r'\s+', ' ', regex=True)         # Replace multiple spaces with
        ↪a single space
        .str.lower()                                # Convert all text to lowercase
    )

# Preview cleaned text columns
df[text_columns].head()

```

```

[25]:
      location          type payment_method
0  swan lake gouna, al gouna, hurghada, red sea  chalet      cash
1  karmell, new zayed city, sheikh zayed city, giza  villa      cash
2  azha north, ras al hekma, north coast  chalet      cash
3  taj city, 5th settlement compounds, the 5th se... apartment  installments
4  granville, new capital city, cairo  villa      cash

```

4.8 Step 7: Standardize categorical column values

```
[26]: ## Step 5: Normalize Categorical Columns (Standardize Property Types & Payment_
      ↪Methods)

      # --- Normalize property types ---
      # Group similar property types together to reduce category fragmentation
      df['type'] = df['type'].replace({
          'apartment': 'apartment',
          'duplex': 'apartment',
          'penthouse': 'apartment',
          'ivilla': 'apartment',

          'villa': 'villa',
          'twin house': 'villa',
          'townhouse': 'villa',

          'chalet': 'chalet',
          'bungalow': 'chalet',
          'cabin': 'chalet',

          'land': 'land',

          # Any other rare or unclear types grouped as "other"
          'palace': 'other',
          'bulk sale unit': 'other',
          'whole building': 'other',
          'full floor': 'other',
          'roof': 'other',
          'hotel apartment': 'other',
          'unknown': 'other'
      })

      # --- Normalize payment method values ---
      df['payment_method'] = df['payment_method'].replace({
          'cash': 'cash',
          'installments': 'installments'
      })

      # --- Verify normalization ---
      print("Unique property types after normalization:\n", df['type'].unique())
      print("\nUnique payment methods after normalization:\n", df['payment_method'].
      ↪unique())
```

Unique property types after normalization:
['chalet' 'villa' 'apartment' 'other' 'land']

Unique payment methods after normalization:

```
['cash' 'installments']
```

4.9 Step 8: Handle Outliers

```
[27]: ## Step 6: Handle Outliers in Numeric Columns (Capping Method)
```

```
# Define numeric columns
numeric_columns = ['price', 'size_sqm', 'down_payment']

# Apply capping between the 1st and 99th percentiles
for col in numeric_columns:
    lower = df[col].quantile(0.01)
    upper = df[col].quantile(0.99)
    df[col] = df[col].clip(lower, upper)
    print(f"{col}: capped between {lower:.2f} and {upper:.2f}")

# Verify after capping
df[numeric_columns].describe()
```

```
price: capped between 1450000.00 and 74240625.00
```

```
size_sqm: capped between 45.00 and 723.54
```

```
down_payment: capped between 0.00 and 9000000.00
```

```
[27]:
```

	price	size_sqm	down_payment
count	1.992400e+04	19924.000000	1.992400e+04
mean	1.447986e+07	201.484263	1.514457e+06
std	1.329897e+07	120.904962	1.617173e+06
min	1.450000e+06	45.000000	0.000000e+00
25%	6.000000e+06	125.000000	5.640000e+05
50%	1.020000e+07	170.000000	1.000000e+06
75%	1.750000e+07	238.000000	1.795000e+06
max	7.424063e+07	723.540000	9.000000e+06

5 Step 9: Clean and split location column

```
[28]: if 'location' in df.columns:
```

```
    # location ): "karmell, new zayed city, sheikh zayed city,
    ↪giza")
    location_split = df['location'].str.split(',', expand=True)

    # 3 ) descriptive (
    df['area'] = location_split[0].str.strip().str.title()
    df['city'] = location_split[1].str.strip().str.title()
    df['governorate'] = location_split[2].str.strip().str.title()
```

```

#
df['city'] = df['city'].fillna('Unknown')
df['governorate'] = df['governorate'].fillna('Unknown')

#      location
df = df.drop(columns=['location'], errors='ignore')

else:
    print(" No 'location' column found - skipping location cleaning.")

```

5.1 Step 10: Save the cleaned dataset

```

[29]: df.to_csv("cleaned_dataset.csv", index=False, sep=",", quotechar='"',
        encoding="utf-8")

print(" Clean dataset saved successfully as 'cleaned_real_estate.csv'")
print("Final shape:", df.shape)

```

Clean dataset saved successfully as 'cleaned_real_estate.csv'
Final shape: (19924, 13)

6 Revision

```

[30]: from ydata_profiling import ProfileReport

profile = ProfileReport(df, title="data Report", explorative=True)

profile.to_notebook_iframe()

```

```

Summarize dataset:  0%|          | 0/5 [00:00<?, ?it/s]
100%|          | 13/13 [00:00<00:00, 28.91it/s]
Generate report structure:  0%|          | 0/1 [00:00<?, ?it/s]
Render HTML:  0%|          | 0/1 [00:00<?, ?it/s]
<IPython.core.display.HTML object>

```

```

[31]: df.isnull().sum()

```

```

[31]: price          0
      type          0
      bedrooms      0
      bathrooms     0
      payment_method 0
      down_payment   0
      size_sqm       0

```



```

down_payment_ratio    0
year                  0
month                 0
area                  0
city                  0
governorate           0
dtype: int64

```

```
[32]: df.head()
```

```

[32]:      price      type  bedrooms  bathrooms  payment_method  down_payment
size_sqm  down_payment_ratio  year \
0   8000000.0    chalet      3.0        1.0        cash      1200000.0
68.0          0.15    2025
1  25000000.0    villa      4.0        4.0        cash      2100000.0
220.0          0.08    2025
2  15135000.0    chalet      2.0        2.0        cash      1513000.0
118.0          0.10    2025
3  12652000.0  apartment      3.0        2.0  installments      1260000.0
166.0          0.10    2025
4  45250000.0    villa      7.0        7.0        cash      2262500.0
400.0          0.05    2025

```

```

      month      area      city      governorate
0   August  Swan Lake Gouna      Al Gouna      Hurghada
1  September      Karmell      New Zayed City  Sheikh Zayed City
2   August      Azha North      Ras Al Hekma      North Coast
3   August      Taj City  5Th Settlement Compounds  The 5Th Settlement
4  September      Granville      New Capital City      Cairo

```

```
[33]: df.info()
display(df.describe(include='all').T)
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19924 entries, 0 to 19923
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   price               19924 non-null  float64
1   type                19924 non-null  object
2   bedrooms            19924 non-null  float64
3   bathrooms           19924 non-null  float64
4   payment_method      19924 non-null  object
5   down_payment        19924 non-null  float64
6   size_sqm            19924 non-null  float64
7   down_payment_ratio  19924 non-null  float64
8   year                19924 non-null  Int64

```

```

9   month                19924 non-null object
10  area                  19924 non-null object
11  city                  19924 non-null object
12  governorate           19924 non-null object
dtypes: Int64(1), float64(6), object(6)
memory usage: 2.0+ MB

```

	count	unique	top	freq	
↳mean	std	min \			
price	19924.0	NaN	NaN	NaN	14479857.
↳393295	13298974.193465	1450000.0			
type	19924	5	apartment	9814	
↳NaN	NaN	NaN			
bedrooms	19924.0	NaN	NaN	NaN	2.
↳891387	0.687216	1.0			
bathrooms	19924.0	NaN	NaN	NaN	2.
↳921351	1.23355	1.0			
payment_method	19924	2	cash	16062	
↳NaN	NaN	NaN			
down_payment	19924.0	NaN	NaN	NaN	1514456.
↳596592	1617172.570581	0.0			
size_sqm	19924.0	NaN	NaN	NaN	201.
↳484263	120.904962	45.0			
down_payment_ratio	19924.0	NaN	NaN	NaN	0.
↳111483	0.07429	0.0			
year	19924.0	<NA>	<NA>	<NA>	2025.
↳004316	0.123451	2023.0			
month	19924	12	August	11204	
↳NaN	NaN	NaN			
area	19924	1469	Marassi	433	
↳NaN	NaN	NaN			
city	19924	296	5Th Settlement Compounds	3670	
↳NaN	NaN	NaN			
governorate	19924	81	North Coast	4721	
↳NaN	NaN	NaN			

	25%	50%	75%	max
price	6000000.0	10200000.0	17500000.0	74240625.0
type	NaN	NaN	NaN	NaN
bedrooms	3.0	3.0	3.0	7.0
bathrooms	2.0	3.0	4.0	7.0
payment_method	NaN	NaN	NaN	NaN
down_payment	564000.0	1000000.0	1795000.0	9000000.0
size_sqm	125.0	170.0	238.0	723.54
down_payment_ratio	0.1	0.1	0.1	0.5
year	2025.0	2025.0	2025.0	2027.0
month	NaN	NaN	NaN	NaN

area	NaN	NaN	NaN	NaN
city	NaN	NaN	NaN	NaN
governorate	NaN	NaN	NaN	NaN

7 Exploratory Data Analysis (EDA)

7.1 Step 1: General Overview

```
[34]: print("Shape of DataFrame:", df.shape)
print("\nData Types:\n", df.dtypes)
display(df.head())
display(df.describe().T)
print("\nMissing Values per Column:\n", df.isnull().sum())
```

Shape of DataFrame: (19924, 13)

Data Types:

price	float64
type	object
bedrooms	float64
bathrooms	float64
payment_method	object
down_payment	float64
size_sqm	float64
down_payment_ratio	float64
year	Int64
month	object
area	object
city	object
governorate	object

dtype: object

	price	type	bedrooms	bathrooms	payment_method	down_payment	size_sqm	down_payment_ratio	year
0	8000000.0	chalet	3.0	1.0	cash	1200000.0	68.0	0.15	2025
1	25000000.0	villa	4.0	4.0	cash	2100000.0	220.0	0.08	2025
2	15135000.0	chalet	2.0	2.0	cash	1513000.0	118.0	0.10	2025
3	12652000.0	apartment	3.0	2.0	installments	1260000.0	166.0	0.10	2025
4	45250000.0	villa	7.0	7.0	cash	2262500.0	400.0	0.05	2025

	month	area	city	governorate
0	August	Swan Lake Gouna	Al Gouna	Hurghada

1	September	Karmell		New Zayed City	Sheikh Zayed City
2	August	Azha North		Ras Al Hekma	North Coast
3	August	Taj City	5Th Settlement Compounds	The 5Th Settlement	
4	September	Granville		New Capital City	Cairo

	count	mean	std	min	
↪25%	50%	75% \			
price	19924.0	14479857.393295	13298974.193465	1450000.0	↪
↪6000000.0	10200000.0	17500000.0			
bedrooms	19924.0	2.891387	0.687216	1.0	↪
↪3.0	3.0	3.0			
bathrooms	19924.0	2.921351	1.23355	1.0	↪
↪2.0	3.0	4.0			
down_payment	19924.0	1514456.596592	1617172.570581	0.0	↪
↪564000.0	1000000.0	1795000.0			
size_sqm	19924.0	201.484263	120.904962	45.0	↪
↪125.0	170.0	238.0			
down_payment_ratio	19924.0	0.111483	0.07429	0.0	↪
↪0.1	0.1	0.1			
year	19924.0	2025.004316	0.123451	2023.0	↪
↪2025.0	2025.0	2025.0			

	max
price	74240625.0
bedrooms	7.0
bathrooms	7.0
down_payment	9000000.0
size_sqm	723.54
down_payment_ratio	0.5
year	2027.0

Missing Values per Column:

price	0
type	0
bedrooms	0
bathrooms	0
payment_method	0
down_payment	0
size_sqm	0
down_payment_ratio	0
year	0
month	0
area	0
city	0
governorate	0
dtype: int64	

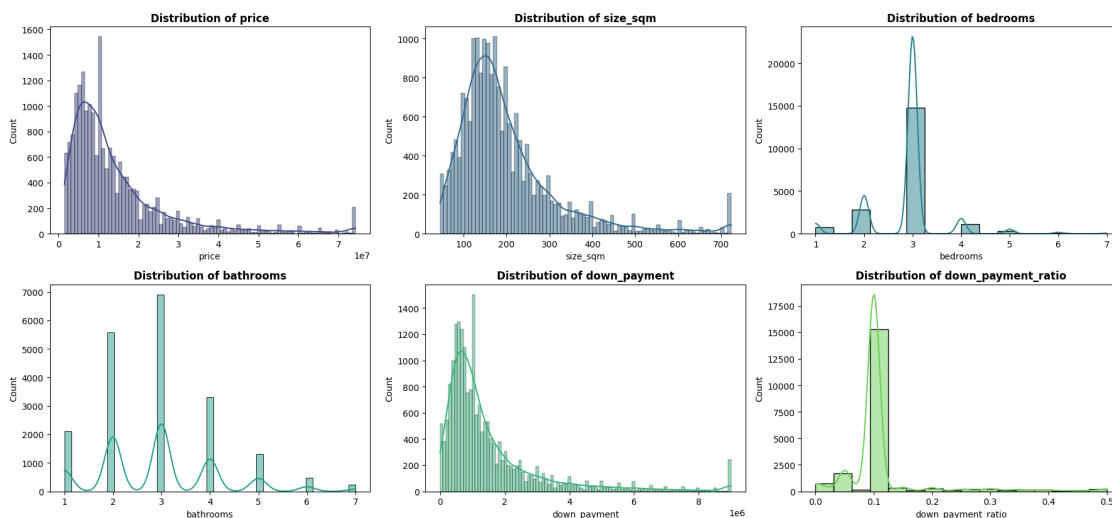
7.2 Step 2: Analyze Numeric Features (Distributions & Outliers)

```
[35]: import matplotlib.pyplot as plt
import seaborn as sns

[36]: numeric_cols = ['price', 'size_sqm', 'bedrooms', 'bathrooms', 'down_payment', 'down_payment_ratio']

plt.figure(figsize=(18, 12))
palette = sns.color_palette("coolwarm", as_cmap=True) #

for i, col in enumerate(numeric_cols, 1):
    plt.subplot(3, 3, i)
    sns.histplot(df[col], kde=True, color=sns.color_palette("viridis", 8)[i % 8], edgecolor="black")
    plt.title(f"Distribution of {col}", fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```



7.3 Step 3: Analyze Categorical Features

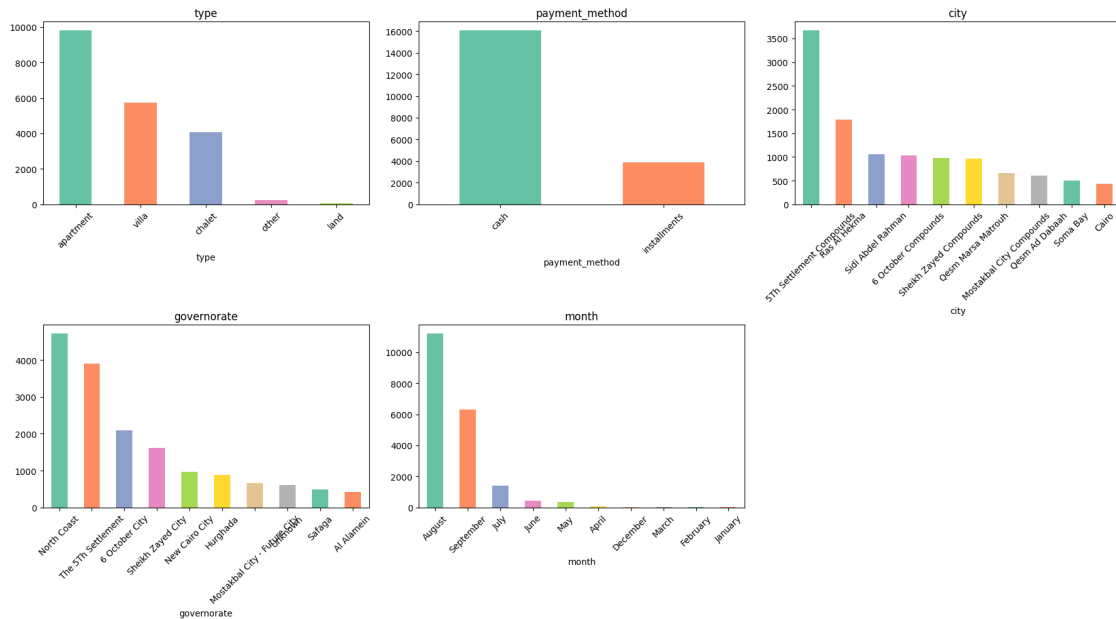
```
[37]: categorical_cols = ['type', 'payment_method', 'city', 'governorate', 'month']

plt.figure(figsize=(18, 10))

for i, col in enumerate(categorical_cols, 1):
    plt.subplot(2, 3, i)
    df[col].value_counts().head(10).plot(kind='bar', color=sns.
    color_palette('Set2'))
```

```
plt.title(col)
plt.xticks(rotation=45)

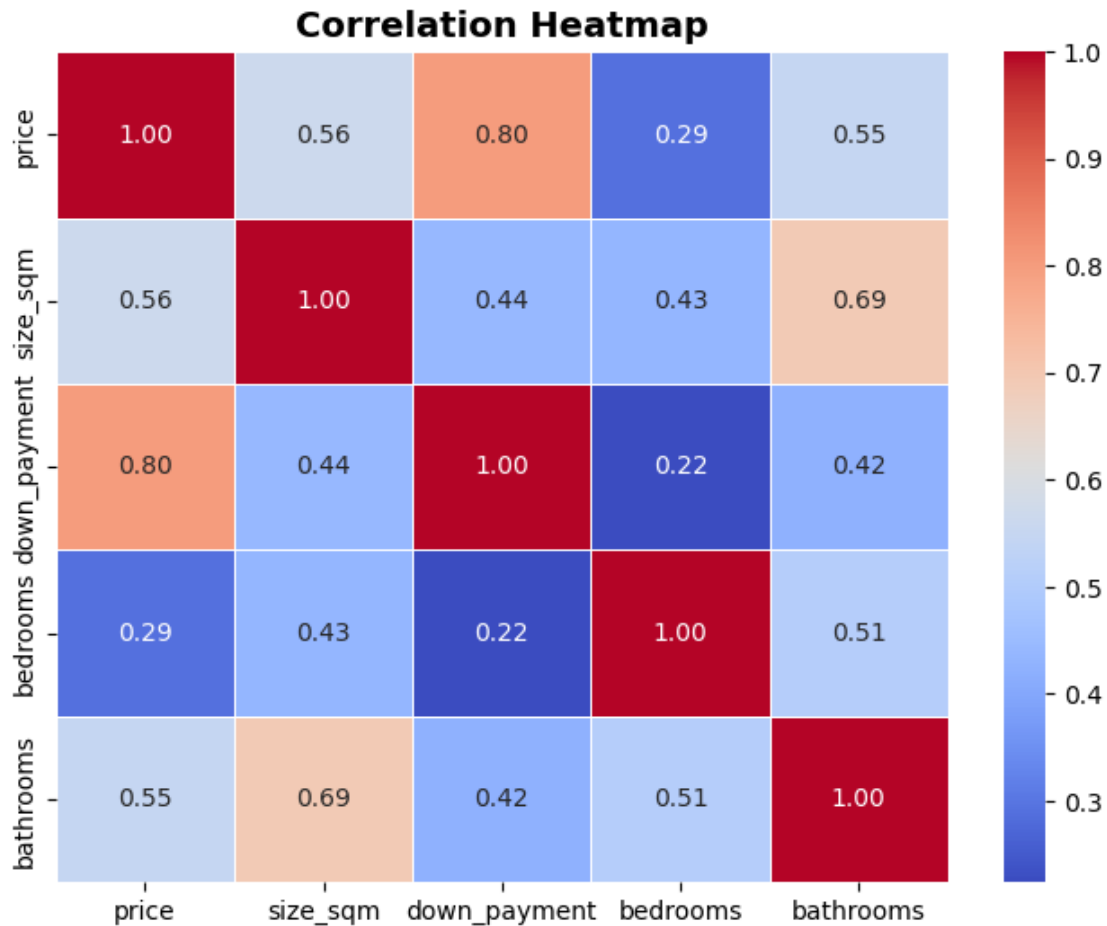
plt.tight_layout()
plt.show()
```



7.4 Step 4: Correlation Between Numeric Features

```
[38]: corr = df[['price', 'size_sqm', 'down_payment', 'bedrooms', 'bathrooms']].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap", fontsize=14, fontweight='bold')
plt.show()
```



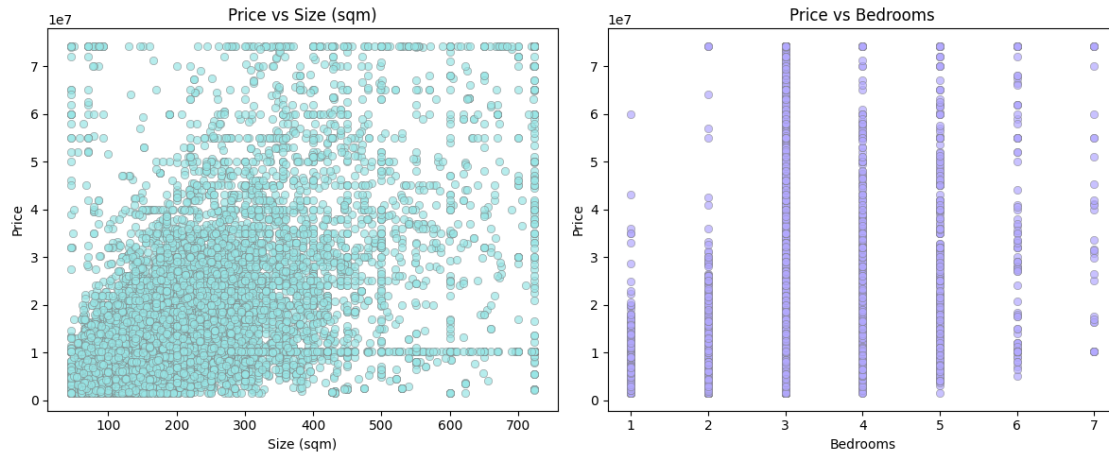
7.5 Step 5: Relationship Between Numeric Variables

```
[39]: plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
sns.scatterplot(x='size_sqm', y='price', data=df, color='#99e6e6', alpha=0.7,
               edgecolor='gray')
plt.title('Price vs Size (sqm)'); plt.xlabel('Size (sqm)'); plt.ylabel('Price')

plt.subplot(1,2,2)
sns.scatterplot(x='bedrooms', y='price', data=df, color='#b3a9ff', alpha=0.7,
               edgecolor='gray')
plt.title('Price vs Bedrooms'); plt.xlabel('Bedrooms'); plt.ylabel('Price')

plt.tight_layout(); plt.show()
```



7.6 Step 6: Analyze Patterns — Average Prices by City

7.6.1 Identify which cities have the highest and lowest average property prices

```
[40]: avg_price_city = df.groupby('city')['price'].mean().
      ↪sort_values(ascending=False).head(15)

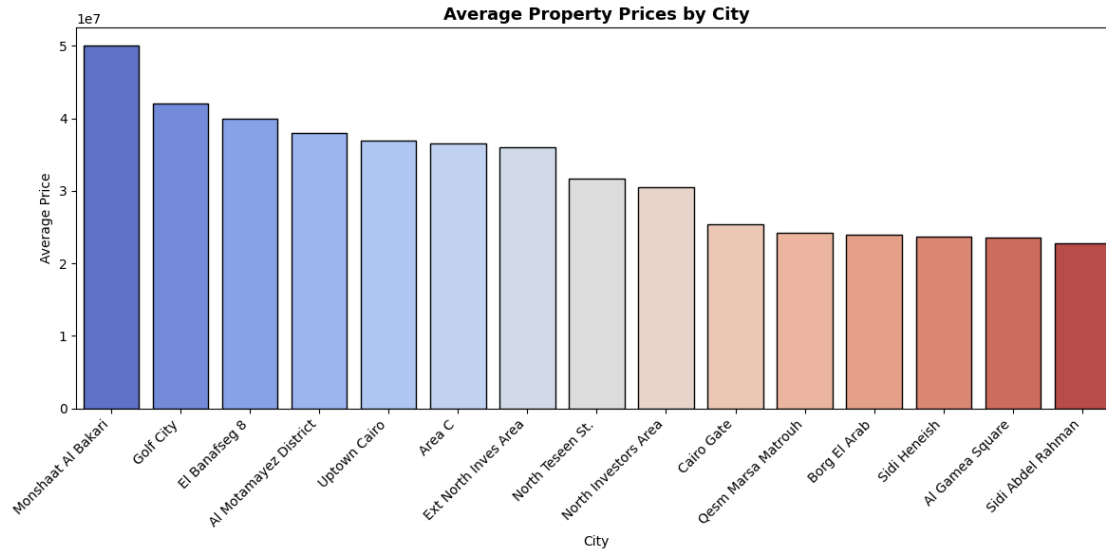
plt.figure(figsize=(12, 6))
sns.barplot(x=avg_price_city.index, y=avg_price_city.values,
            ↪palette='coolwarm', edgecolor='black')
plt.title('Average Property Prices by City', fontsize=13, fontweight='bold')
plt.xlabel('City')
plt.ylabel('Average Price')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\460487364.py:4:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=avg_price_city.index, y=avg_price_city.values,
            palette='coolwarm', edgecolor='black')
```

7.7 Step 7: Analyze Patterns — Average Prices by Property Type

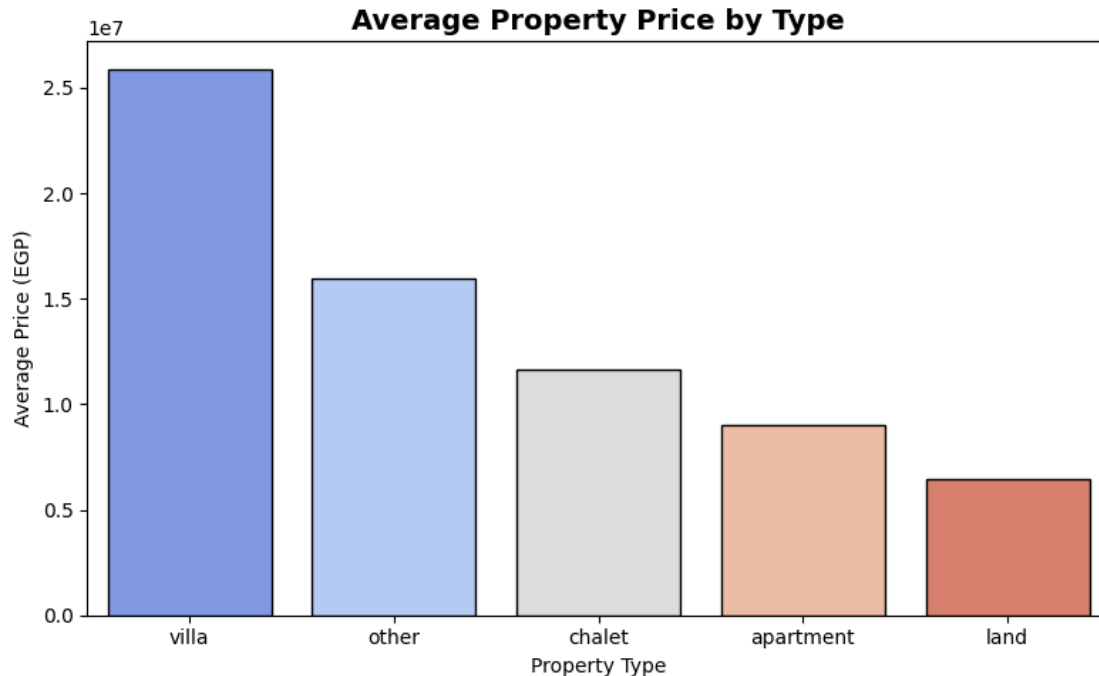
```
[41]: avg_price_type = df.groupby('type')['price'].mean().sort_values(ascending=False)

plt.figure(figsize=(8, 5))
sns.barplot(
    x=avg_price_type.index,
    y=avg_price_type.values,
    palette="coolwarm",
    edgecolor="black"
)
plt.title("Average Property Price by Type", fontsize=14, fontweight="bold")
plt.xlabel("Property Type")
plt.ylabel("Average Price (EGP)")
plt.tight_layout()
plt.show()
```

C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\1037578925.py:4:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



7.8 Step 8: Average Price by Payment Method

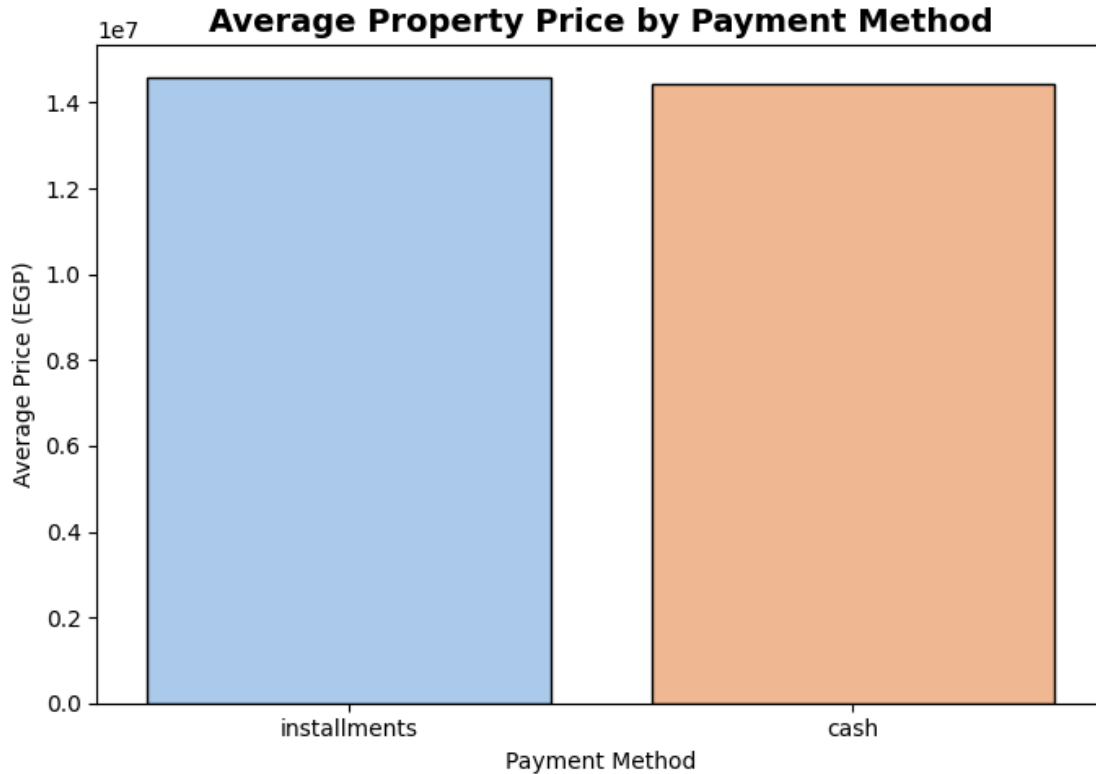
```
[42]: avg_price_payment = df.groupby('payment_method')['price'].mean().
      ↪sort_values(ascending=False)

plt.figure(figsize=(7, 5))
sns.barplot(
    x=avg_price_payment.index,
    y=avg_price_payment.values,
    palette="pastel",
    edgecolor="black"
)
plt.title("Average Property Price by Payment Method", fontsize=14,
      ↪fontweight="bold")
plt.xlabel("Payment Method")
plt.ylabel("Average Price (EGP)")
plt.tight_layout()
plt.show()
```

C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\3048645520.py:4:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



7.9 Step 9: Analyze Property Availability Over Time

7.9.1 Understand how property listings and prices vary by month and year

```
[43]: plt.figure(figsize=(12,5))
sns.barplot(x='month', y='price', data=df, palette='Pastel2', estimator='mean',
            edgecolor='black')
plt.title('Average Property Price by Month', fontsize=13, fontweight='bold')
plt.xlabel('Month')
plt.ylabel('Average Price')
plt.tight_layout()
plt.show()

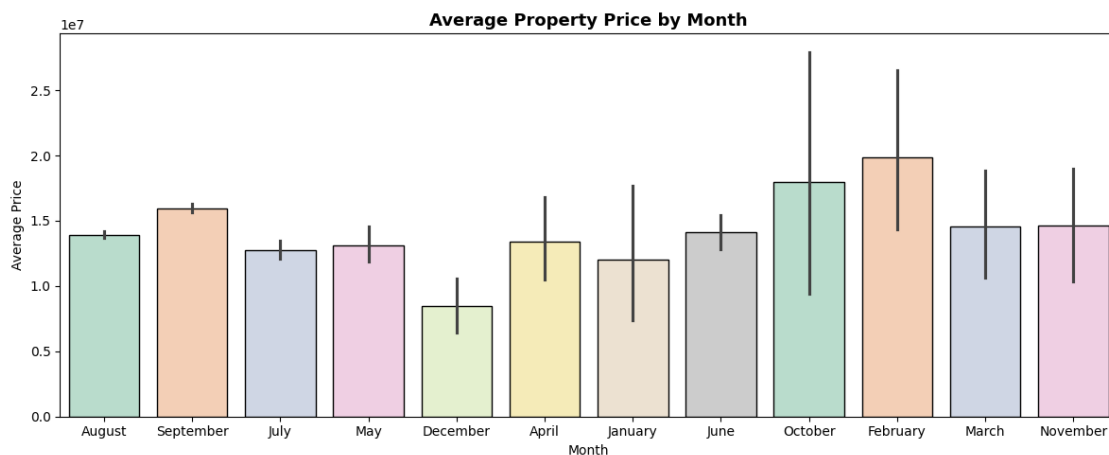
plt.figure(figsize=(12,5))
sns.barplot(x='year', y='price', data=df, palette='Pastel2', estimator='mean',
            edgecolor='black')
plt.title('Average Property Price by Year', fontsize=13, fontweight='bold')
plt.xlabel('Year')
plt.ylabel('Average Price')
```

```
plt.tight_layout()
plt.show()
```

C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\1786970075.py:2:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

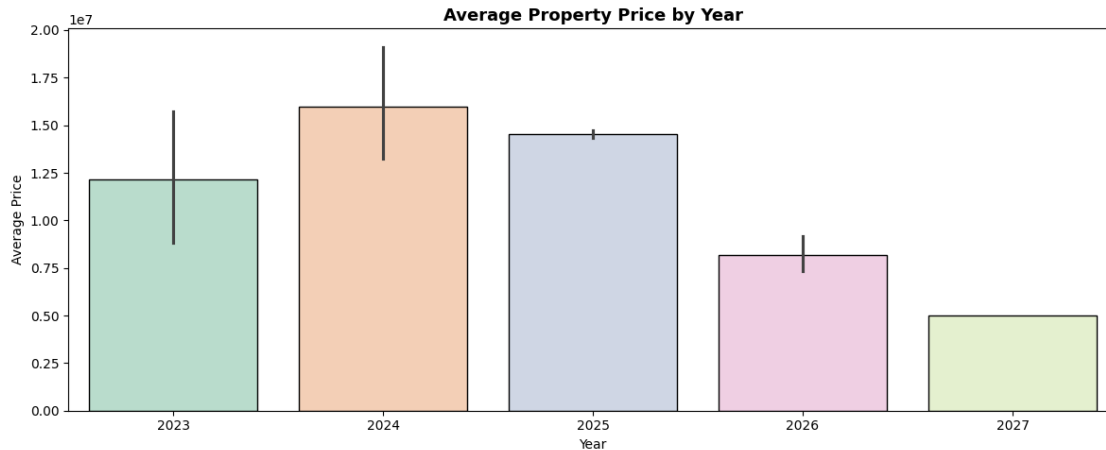
```
sns.barplot(x='month', y='price', data=df, palette='Pastel2',
estimator='mean', edgecolor='black')
```



C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\1786970075.py:10:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='year', y='price', data=df, palette='Pastel2', estimator='mean',
edgecolor='black')
```



7.10 Step 10: Property Availability by Month and Year

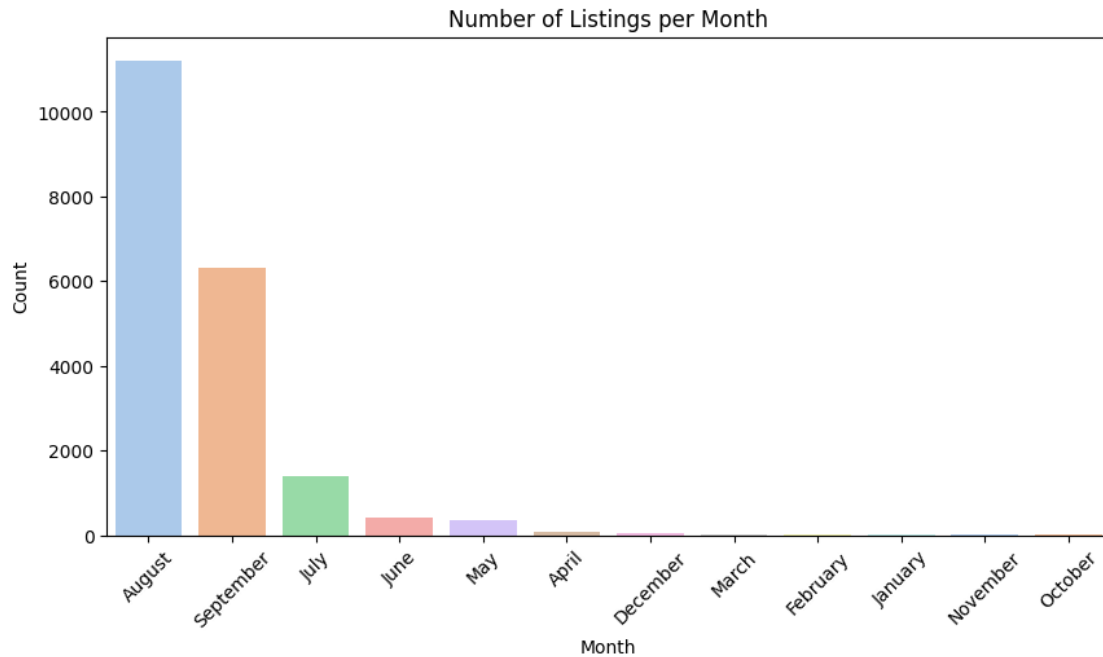
```
[44]: plt.figure(figsize=(10, 5))
sns.countplot(x='month', data=df, palette='pastel', order=df['month'].
    ↳value_counts().index)
plt.title('Number of Listings per Month')
plt.xlabel('Month')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(6, 4))
sns.countplot(x='year', data=df, palette='cool')
plt.title('Number of Listings per Year')
plt.xlabel('Year')
plt.ylabel('Count')
plt.show()
```

C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\2441227480.py:2:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='month', data=df, palette='pastel',
order=df['month'].value_counts().index)
```

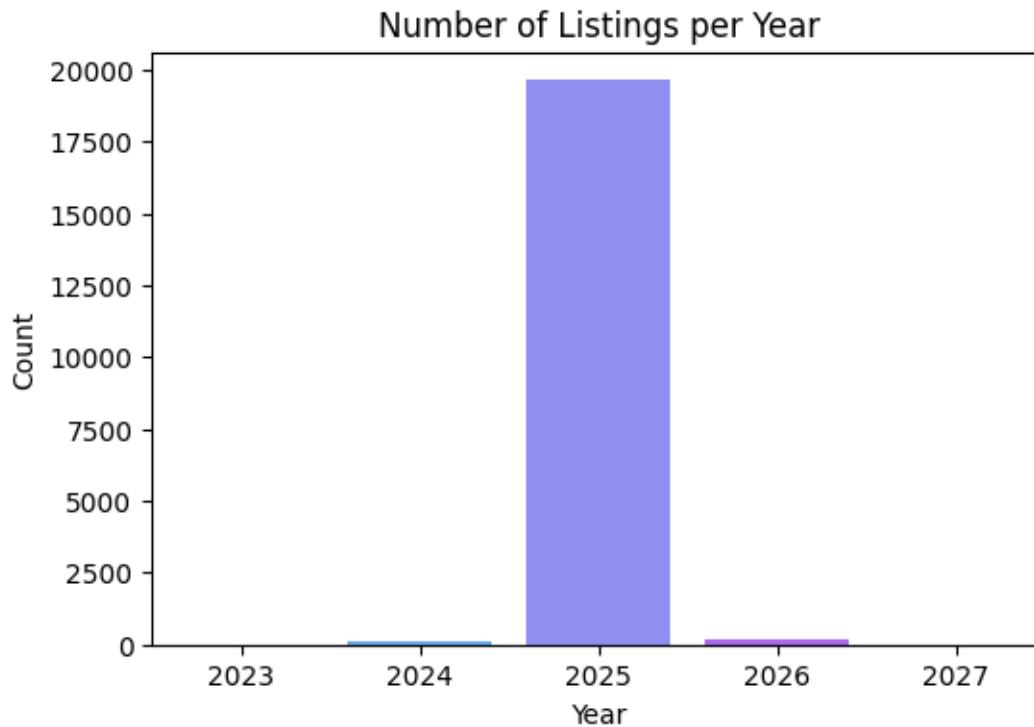


C:\Users\FIRST COMPUTER\AppData\Local\Temp\ipykernel_6540\2441227480.py:10:

FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

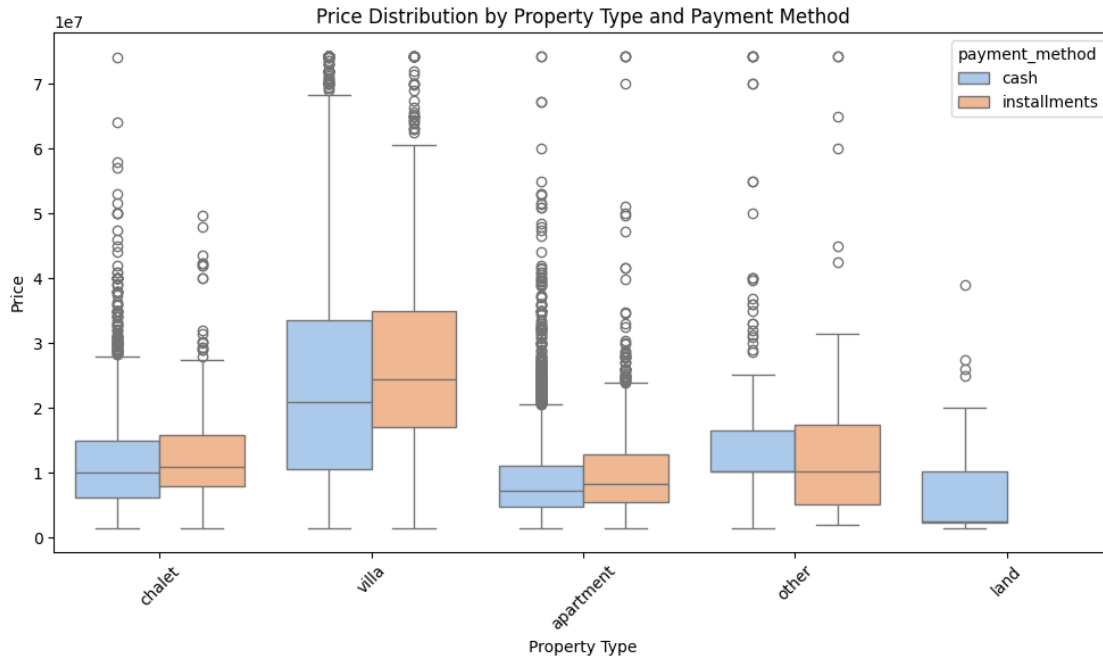
```
sns.countplot(x='year', data=df, palette='cool')
```



7.11 Step 11: Analyze Price by Property Type and Payment Method

7.11.1 Compare how price varies across property types for each payment method

```
[45]: plt.figure(figsize=(10, 6))
sns.boxplot(x='type', y='price', hue='payment_method', data=df,
            palette='pastel')
plt.title('Price Distribution by Property Type and Payment Method')
plt.xlabel('Property Type')
plt.ylabel('Price')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



7.12 Summary & Insights

7.12.1 Key findings and observations from the EDA

```
[46]: # 1. General numerical summary
print(" Basic Statistics for Numeric Columns:\n")
print(df.describe().T)

# 2. Identify top 5 most expensive cities
top_cities = df.groupby('city')['price'].mean().sort_values(ascending=False).
    ↪head(5)
print("\n Top 5 Cities by Average Price:\n", top_cities)

# 3. Property type with highest average price
top_type = df.groupby('type')['price'].mean().sort_values(ascending=False).
    ↪head(3)
print("\n Property Types with Highest Average Price:\n", top_type)

# 4. Payment method trends
payment_trend = df.groupby('payment_method')['price'].mean().
    ↪sort_values(ascending=False)
print("\n Average Price by Payment Method:\n", payment_trend)

# 5. Availability trend
month_counts = df['month'].value_counts().sort_index()
```



```
print("\n Property Listings by Month:\n", month_counts)
```

Basic Statistics for Numeric Columns:

	count	mean	std	min
price	19924.0	14479857.393295	13298974.193465	1450000.0
bedrooms	19924.0	2.891387	0.687216	1.0
bathrooms	19924.0	2.921351	1.23355	1.0
down_payment	19924.0	1514456.596592	1617172.570581	0.0
size_sqm	19924.0	201.484263	120.904962	45.0
down_payment_ratio	19924.0	0.111483	0.07429	0.0
year	19924.0	2025.004316	0.123451	2023.0

	max
price	74240625.0
bedrooms	7.0
bathrooms	7.0
down_payment	9000000.0
size_sqm	723.54
down_payment_ratio	0.5
year	2027.0

Top 5 Cities by Average Price:

city	
Monshaat Al Bakari	5.000000e+07
Golf City	4.200000e+07
El Banafseg 8	4.000000e+07
Al Motamayez District	3.797316e+07
Uptown Cairo	3.696256e+07

Name: price, dtype: float64

Property Types with Highest Average Price:

type	
villa	2.589603e+07
other	1.593053e+07
chalet	1.165456e+07

Name: price, dtype: float64

Average Price by Payment Method:

```
payment_method
installments    1.460461e+07
cash            1.444986e+07
Name: price, dtype: float64
```

```
Property Listings by Month:
month
April          83
August        11204
December       42
February       25
January        22
July          1385
June           418
March           30
May            360
November       20
October        19
September     6316
Name: count, dtype: int64
```

8 Machine Learning Regression Model

8.0.1 • Target Variable: The variable we want to predict

8.1 Define features and target

```
[47]: features = ['size_sqm', 'bedrooms', 'bathrooms', 'down_payment',
                'type', 'payment_method', 'city', 'governorate']
      target = 'price'
```

8.2 Split data & Encode categorical variables

```
[48]: # Encode categorical variables
      df_encoded = pd.get_dummies(df[features], drop_first=True)
      X = df_encoded
      y = df[target]

      # Split data
      X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, random_state=42)

      # Train model
      model = LinearRegression()
      model.fit(X_train, y_train)
      y_pred = model.predict(X_test)
      r2 = r2_score(y_test, y_pred)
```

```

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(" Regression Model Performance:")
print(f"R Score: {r2:.3f}")
print(f"MAE: {mae:,.0f} EGP")
print(f"RMSE: {rmse:,.0f} EGP")

```

```

Regression Model Performance:
R Score: 0.764
MAE: 4,085,302 EGP
RMSE: 6,539,607 EGP

```

8.3 Save model

```

[49]: joblib.dump(model, "price_regression_model.pkl")
print(" Regression model saved!")

```

```

Regression model saved!

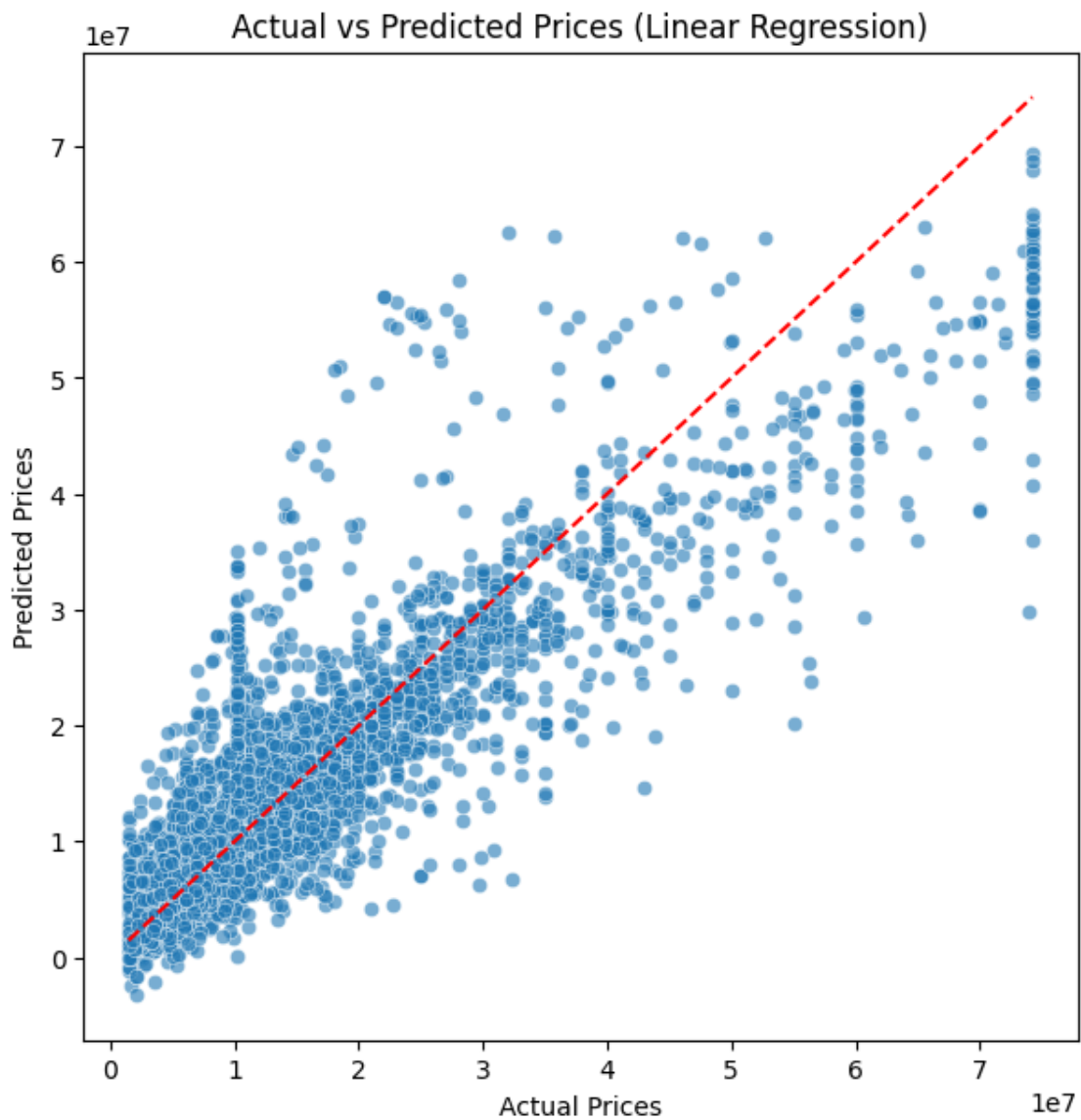
```

8.4 Visualization

```

[50]: plt.figure(figsize=(7,7))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Prices (Linear Regression)")
plt.plot([y.min(), y.max()], [y.min(), y.max()], color='red', linestyle
='--')
plt.show()

```

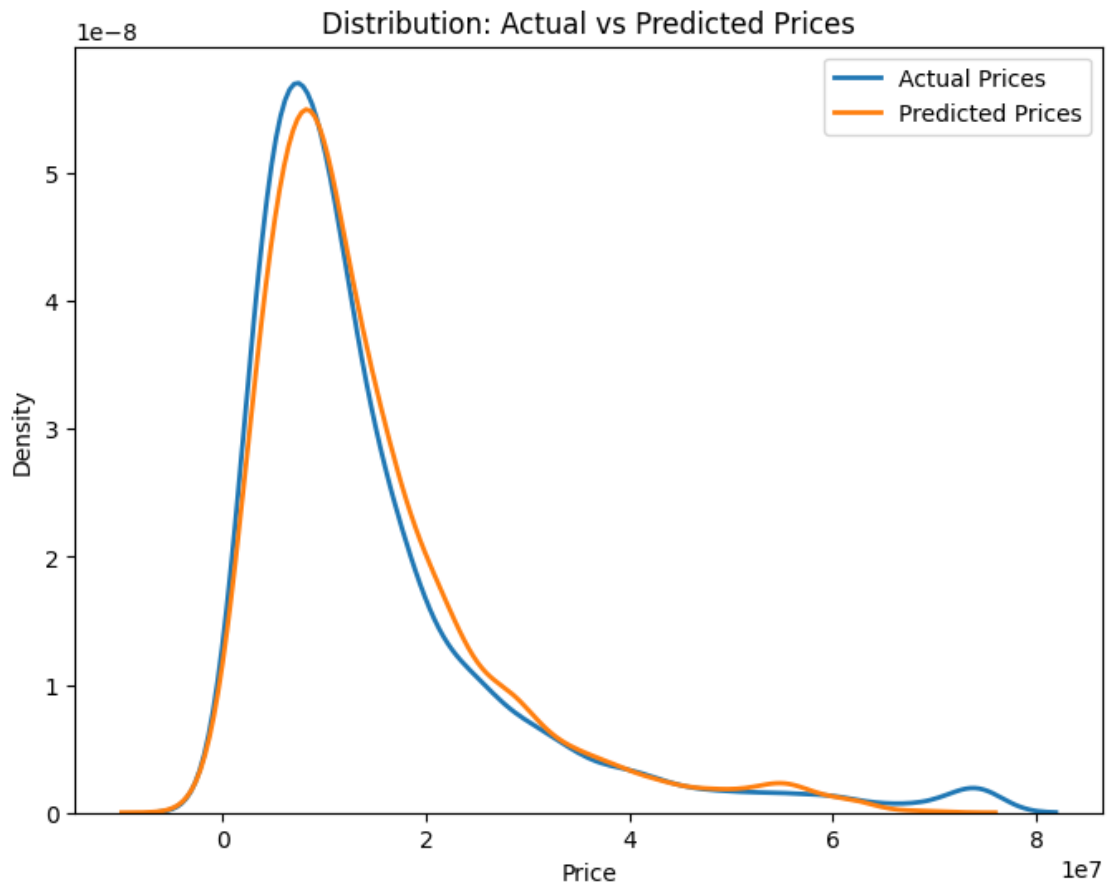


8.5 Distribution of Actual vs Predicted Prices

```
[51]: plt.figure(figsize=(8,6))

sns.kdeplot(y_test, label="Actual Prices", linewidth=2)
sns.kdeplot(y_pred, label="Predicted Prices", linewidth=2)

plt.xlabel("Price")
plt.title("Distribution: Actual vs Predicted Prices")
plt.legend()
plt.show()
```



8.6 Error Distribution Plot (Histogram of Prediction Errors)

```
[52]: errors = y_test - y_pred

plt.figure(figsize=(8,6))
sns.histplot(errors, kde=True, bins=30)
plt.xlabel("Prediction Error (Actual - Predicted)")
plt.title("Distribution of Prediction Errors")
plt.axvline(0, color='red', linestyle='--')
plt.show()
```

